Indian Institute of Technology Kharagpur

Department of Computer Science and Engineering

High QPS Text Search Engine

Project Velocity

Course: CS30202: Database Management Systems

Term Project Report

Spring Semester 2025

GitHub Repository:

https://github.com/Dheeraj-yellapu/project-velocity.git

Datasets used:

https://github.com/Dheeraj-yellapu/project-velocity/blob/main/database_url.md

Group Name	Query_Ninjas
B. Siddaarth	23CS30011
Yellapu Dheeraj Babu	23CS10081
Maddipatla Sai Sravan	23CS10041
Garlapati Srimannadh Reddy	23CS10020

Contents

1 Introduction **4**

I Project Overview . . . 4

II Problem Statement . . . 4

III Goals and Objectives . . . 4

IV Technology Stack . . . 5

2 Methodology **6**

I System Architecture . . . 6

II Dataset and Indexing . . . 6

III Benchmarking Approach . . . 7

IV Deployment Configuration . . . 7

3 Features Implemented **8**

I Role-Based Landing Page . . . 8

II Solr Schema Design . . . 8

III Core Full-Text Search with BM25 and EDISMax . . . 8

III.I Relevance Ranking . . . 8

III.II Per-Query Performance Metrics . . . 9

III.III Tradeoffs . . . 9

IV Faceted Filtering and Sort Options . . . 9

V Auto-Suggestions (Type-Ahead Search) . . . 9

VI Three-Tier Caching Architecture . . . 10

VII Nginx Reverse Proxy and Load Balancing . . . 10

VIII SolrCloud Distributed Deployment with ZooKeeper . . . 10

IX Security Middleware Stack . . . 10

X Query Analytics Dashboard (Admin Overview) . . . 11

XI Live Query Logs . . . 11

XII System Metrics and Node Monitoring . . . 11

XIII Internal Benchmarking Endpoint . . . 12

XIV Apache Airflow ETL Pipeline . . . 12

XV Docker Compose Orchestration . . . 12

4 Experiments and Results **13**

I Internal Cache Benchmark Results . . . 13

II Cache Effectiveness Test . . . 13

III Throughput Experiments . . . 14

IV Mixed Query Load Test . . . 15

IV.I Run 1: 10 Query Terms, 300 Requests . . . 15

IV.II Run 2: 42 Query Terms, 9,448 Requests (Stress Test) 15
IV.III Run 3: 42 Query Terms, 1,000 Requests (Optimised) 16
IV.IV Run 4: 42 Query Terms, 10,000 Requests (Peak Performance) 16
V Summary of Experimental Results 16

5 Screenshots **17**
I Role-Based Landing Page 17
II Search Interface — Solr vs. Cache Comparison 17
III Full Search Interface with Faceted Filters 18
IV Search Results — Direct Solr Query 18
V Admin Dashboard — Overview Panel 19
VI Admin Dashboard — Live Query Logs 19
VII Admin Dashboard — Prometheus/Grafana-Style Profiling 20
VIII Admin Dashboard — Performance and Benchmark 21
IX Admin Dashboard — Settings Panel 21
X Apache Airflow ETL Pipeline 22

6 Conclusion **23**
I Summary of Achievements 23
II Challenges Encountered 23
III Future Scope 23

7 References **25**

Chapter 1

Introduction

I Project Overview

Project Velocity is a high-throughput, distributed text search engine designed to index and serve large volumes of news articles with minimal latency under concurrent query load. The system is built on top of **Apache Solr 9.4.1** for distributed full-text indexing and **Apache ZooKeeper 3.8** for cluster coordination, backed by a **Node.js/Express** API layer, a **Redis 7** caching tier, and a **React.js 18** frontend. The core motivation is to evaluate how modern distributed search infrastructures handle high Queries Per Second (QPS) workloads while maintaining sub-100 ms average response latencies.

Over five weeks, we designed and deployed a multi-node SolrCloud topology across four physical machines; implemented a three-tier caching architecture with a single-flight request-coalescing mechanism achieving an average 82× speedup; built a comprehensive security middleware stack; developed an Apache Airflow ETL pipeline for automated dataset ingestion; and delivered a feature-rich React-based admin dashboard with real-time analytics and exportable benchmark reports. The system was successfully benchmarked at **1,000 QPS** with 10,000 concurrent queries and zero error rate.

II Problem Statement

Full-text search over large document corpora is a fundamental capability required by news portals, digital libraries, and enterprise search platforms. The challenge lies in sustaining high query throughput under concurrent load while preserving relevance ranking quality and maintaining low end-to-end latency. This project investigates the design trade-offs inherent in constructing such a system using open-source distributed search technologies.

III Goals and Objectives

1. Deploy a **SolrCloud cluster** across four physical machines, coordinated by ZooKeeper, capable of horizontal scaling through shards and replicas.
2. Achieve and benchmark **1000+ QPS** under concurrent load using custom load-testing scripts and Siege.
3. Implement a **three-tier caching architecture** (L1 in-memory, L2 Redis, L3 Solr) with request coalescing to prevent cache stampedes.
4. Provide a **full-featured React search UI** with real-time auto-suggestions, faceted filtering, sort options, and per-query performance metrics.
5. Build a **security middleware stack** with rate limiting, XSS/injection sanitisation, IP-based anomaly detection, and admin authentication.
6. Implement **query analytics and heatmap logging** with time-series QPS charts, latency distributions, and exportable reports.
7. Develop an **Apache Airflow ETL pipeline** for automated dataset extraction, transformation, and indexed loading into SolrCloud.

IV Technology Stack

Layer	**Technology**
Search Engine	Apache Solr 9.4.1 + Apache ZooKeeper 3.8
Backend API	Node.js 20 + Express.js (ES Modules)
Caching	Redis 7 (shared across backend replicas)
Frontend	React.js 18 + Vite + Chart.js + jsPDF + html2canvas
ETL Pipeline	Apache Airflow 2.6 (Python 3)
Reverse Proxy / Load Balancer	Nginx (alpine)
Containerisation	Docker + Docker Compose
Load Testing	Custom Node.js scripts + Siege
Dataset	AG News, MIND, Newsroom (merged via Airflow ETL)

Table 1.1: Technology stack of Project Velocity

Chapter 2

Methodology

I System Architecture

The system follows a layered, distributed architecture comprising five major tiers:

1. **Client Tier:** A React.js single-page application served via Vite. Users interact with the search bar, filter panel, and admin dashboard. A role-selection landing page routes users to either the search interface or the admin dashboard.

2. **Reverse Proxy Tier:** Nginx forwards incoming HTTP requests to backend API nodes using a `split_clients`-based round-robin distribution (33.3% each) and handles response caching at the proxy layer.

3. **Backend API Tier:** Three Node.js/Express replicas (`backend-1`, `backend-2`, `backend-3`), each exposing REST endpoints for search, suggestions, analytics, metrics, and benchmarking.

4. **Caching Tier:** A shared Redis 7 instance provides an L2 distributed cache. Each backend node also maintains an independent L1 in-memory `Map` cache.

5. **Search Index Tier:** A four-node SolrCloud cluster coordinated by ZooKeeper handles distributed indexing, query execution, and replication.

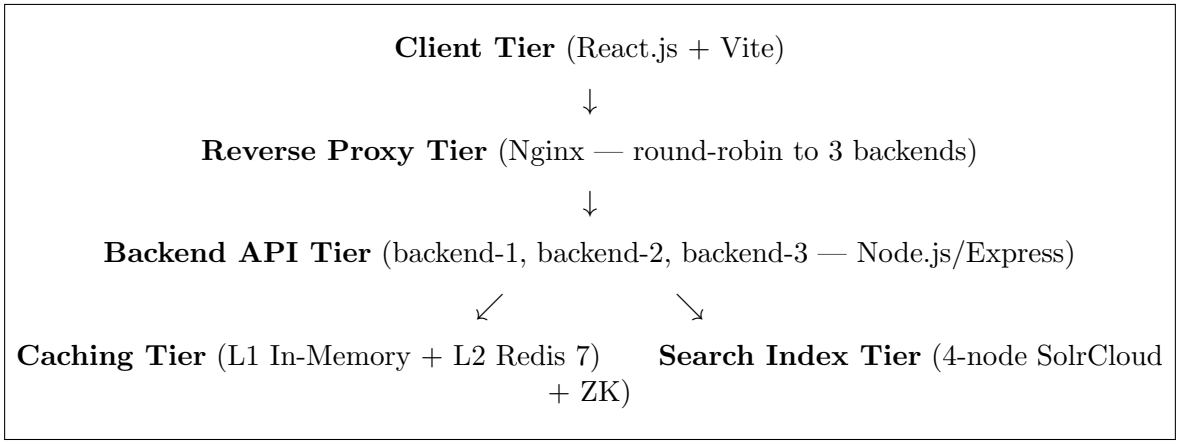


Figure 2.1: Five-tier system architecture of Project Velocity

II Dataset and Indexing

The corpus consists of merged news articles drawn from two publicly available datasets: **News Articles** (main schema dataset) and **News Category Dataset** (used for ETL Pipeline). These datasets are normalised into a common JSON schema with the fields: `id`, `title`, `body`, `sum`, `pub`, `type`, `lang`, and `url`. The merged corpus contains approximately **270,281 documents**, replicated across all four Solr nodes by ZooKeeper.

III Benchmarking Approach

Performance benchmarking was conducted using both custom Node.js load-testing scripts and Siege, configured to simulate concurrent user loads at increasing concurrency levels. The primary metrics recorded are QPS (Queries Per Second), average response latency (ms), throughput (req/s), and error rate (%). An internal `/api/benchmark` endpoint additionally measures the speedup achieved by the Redis and in-memory cache tiers versus direct Solr queries.

IV Deployment Configuration

The entire system is containerised using Docker Compose. The `docker-compose.yml` orchestrates five containers: `shared_redis` (Redis 7, port 6379), three API replicas (`velocity-backend-1/2/3`, ports 4000–4002), and `shared_nginx` (Nginx reverse proxy, port 80). The four SolrCloud nodes are deployed directly on physical machines using shell scripts (`setup_brain.sh` for the leader, `setup_worker.sh` for followers).

Chapter 3

Features Implemented

I Role-Based Landing Page

The application opens with a role-selection landing page presenting two entry points: **User** (for searching the news corpus) and **Admin** (for accessing analytics, logs, and system insights). This separation ensures that regular users are not exposed to administrative controls. The landing page is implemented in `frontend/src/pages/Login.jsx`, rendering two cards with role descriptions. No authentication is required for the user role; the admin role is protected by access-code authentication (Section IX).

II Solr Schema Design

The Solr collection schema (`solr/schema/managed-schema.xml`) is optimised for the specific query patterns of the search engine: full-text retrieval on `title` and `body`, exact faceting on `type` and `lang`, and range filtering on the publication timestamp `pub`. Table 3.1 summarises the key field definitions.

Field	Type	Purpose
`id`	`string`	Unique document identifier
`title`	`text_general`	Article headline; primary search target (weight 5)
`body`	`text_general`	Full article text; lowest weight search field
`sum`	`text_general`	Article summary; secondary search target (weight 2)
`type`	`string`	Topic category (e.g., POLITICS, SPORTS, TECHNOLOGY)
`pub`	`pdate`	Publication date; used for range filtering and recency boost
`lang`	`string`	Language code for language-based filtering
`url`	`string`	Source article URL

Table 3.1: Solr schema field definitions

`text_general` fields use Solr's standard analyser chain: Unicode normalisation, lowercase tokenisation, and stop-word removal, providing built-in query preprocessing and normalisation.

III Core Full-Text Search with BM25 and EDISMax

III.I Relevance Ranking

The search engine supports keyword-based full-text search across all indexed news articles. Relevance ranking is performed by Solr using the **BM25 similarity model**, augmented with the **Extended DisMax (EDISMax)** query parser to enable field-weighted scoring and phrase boosting. Queries are constructed in `backend/src/utils/queryBuilder.js`:

```
const params = {
  q: safeQ,
  defType: "edismax",
  qf: "title^5 type^4 sum^2 body^1",  // Field weights
  pf: "title^10 sum^5",               // Phrase boost
  bf: "recip(ms(NOW,pub),3.16e-11,1,1)", // Recency bias
```

```
  fl: "title,sum,type,pub,body,id,url,lang",
  rows, start,
};
```

Listing 3.1: EDISMax query parameter construction

The `qf` parameter assigns relevance weights: the `title` field carries 5× the weight of `body`. The `pf` parameter boosts documents where query terms appear as a phrase. The `bf` function introduces a recency bias so that more recently published articles receive a slight relevance uplift.

III.II Per-Query Performance Metrics

A distinguishing feature of the search interface is the per-query performance metrics bar displayed above results. For every query, the UI shows: **Total Latency** (ms), **Solr QTime** (ms), **Data Source** (colour-coded — green for "Redis Cache", blue for "Solr Database"), and **Results Found**.

III.III Tradeoffs

The EDISMax parser introduces slightly higher per-query CPU overhead compared to standard DisMax. The recency bias function adds a function query evaluation step, marginally increasing QTime on large result sets. Both tradeoffs are acceptable given the substantially improved relevance.

IV Faceted Filtering and Sort Options

The search interface provides a rich left-hand filter panel enabling users to narrow results across multiple dimensions:

- **Publication Date**: Preset ranges (Any time, Last 24 hours, Last 7 days, Last 30 days, Last year, Custom range).
- **Topic**: Dynamically populated categories fetched via the `/api/topics` facet endpoint.
- **Language**: Multi-language filtering (English, French, German, Spanish).
- **Sort By**: Relevance (BM25 score), Newest First, Oldest First.
- **Results Per Page**: Configurable (10, 20, 50 results).

Filter queries (`fq`) are appended for each active facet. Solr's filter query cache ensures these do not re-score documents on repeated application. Each search result card shows the source domain favicon, topic category badge, article title (linked to original URL), publication date, summary snippet, and an expandable body text.

V Auto-Suggestions (Type-Ahead Search)

As a user types into the search bar, the frontend dispatches debounced prefix-search requests to `/api/suggest`, returning up to ten ranked title suggestions in real time. The `suggestController` issues a Solr wildcard query (`title:prefix*`) against the `title` field using the standard Lucene parser. Fifty candidates are fetched, de-duplicated by title, and the top ten unique suggestions are returned. Results are cached in Redis for five minutes. On the frontend, a 300 ms debounce timer prevents excessive Solr queries.

Tradeoff: The 5-second `timeAllowed` guard ensures queries for very short prefixes are abandoned rather than blocking indefinitely. A production deployment would use Solr's dedicated `Suggester` component for O(prefix-length) lookup.

VI Three-Tier Caching Architecture

To sustain high QPS while minimising Solr load, the system implements a three-tier caching hierarchy with automatic tier promotion and a single-flight (request coalescing) mechanism.

L1 — In-Memory Cache (0 ms): Each backend process maintains a `Map`-based cache with 2-minute TTL and 2,000 max entries. LRU eviction is approximated by removing the first entry at capacity.

L2 — Redis Cache (1–5 ms): The shared Redis 7 instance acts as a distributed L2 cache. On an L1 miss but L2 hit, the result is promoted to L1 for subsequent requests.

L3 — Solr (50–200 ms): A Solr query is only issued when both L1 and L2 miss. Results are populated into both L1 and L2 simultaneously.

Single-Flight (Request Coalescing): The `inflightQueries` Map ensures that when many concurrent requests arrive for the identical query before the cache is populated, only a single Solr request is dispatched. All other in-flight requests share the same `Promise`, preventing cache stampedes.

Tradeoff: The L1 cache is per-process and does not benefit requests routed to different replicas. The shared Redis L2 compensates. The 2-minute TTL means briefly stale results after index updates, acceptable for a news search use case.

VII Nginx Reverse Proxy and Load Balancing

Nginx acts as the ingress point for all API traffic, load-balancing across three backend replicas. The configuration (`docker/nginx.conf`) uses `split_clients` on `$request_id` for equal distribution. Features include: 4,096 worker connections, proxy caching of search results for 10 minutes, `X-Forwarded-For` header propagation for accurate rate limiting, `X-Upstream-Server` header for debugging, and disabled access logging during benchmarks to avoid disk I/O bottlenecks.

VIII SolrCloud Distributed Deployment with ZooKeeper

The search index is distributed across a four-node SolrCloud cluster coordinated by ZooKeeper. Deployment is scripted in `setup_brain.sh` (leader) and `setup_worker.sh` (followers). The brain script starts a ZooKeeper container on port 2181, starts Solr 9 with the cloud flag, and uploads the shared configuration to ZooKeeper using `solr zk upconfig`. The collection is created with four shards and one replica per shard.

The `solrService.js` module uses Axios with custom HTTP agents configured for 500 maximum persistent sockets, reducing TCP connection overhead under high concurrency.

The parameter `shards.info=true` is appended to each query so that response metadata reveals which shard or node served the request.

Tradeoff: A single ZooKeeper instance is a single point of failure. The four-shard, one-replica topology provides no redundancy — a production deployment would use a three-node ZK ensemble with two replicas per shard.

IX Security Middleware Stack

The backend implements a layered security middleware stack:

Rate Limiting: Per-IP rate limiting via `express-rate-limit` restricts the search endpoint to 20 requests per 10-second window per IP, with correct `X-Forwarded-For` handling behind Nginx.

Input Sanitisation: The `sanitizeInputMiddleware` rejects queries exceeding 100 characters,

blocks SQL injection patterns (`UNION SELECT`, `DROP TABLE`), XSS payloads (`<script>`), and expensive Solr patterns (`*:*`, deep wildcards, fuzzy operators).

Anomaly Detection: An IP exceeding 50 requests/minute is automatically blocked for 60 seconds (HTTP 403). Per-IP tracking structures are garbage-collected every 5 minutes.

Audit Logging: Every request is logged asynchronously to Redis, capturing timestamp, IP, query, latency, status, and endpoint. Slow queries ($> 1.5\,\mathrm{s}$) are stored in a 50-entry ring buffer.

Admin Authentication: The admin dashboard is protected by bcrypt-hashed access codes stored in Redis. Administrators can change codes via the Settings panel.

Security Toggle: A global `securityConfig.enabled` flag allows all security middleware to be bypassed during benchmarking runs via the Admin Settings panel.

Tradeoff: Anomaly detection state is per-process and not shared across replicas. A production system would use shared Redis counters for distributed rate tracking.

X Query Analytics Dashboard (Admin Overview)

The admin dashboard (`/admin`) provides real-time and historical analytics. The Overview panel displays five KPIs: current QPS, peak QPS, average latency, throughput, and error rate. A Chart.js line chart renders QPS time-series data bucketed into 15 intervals. A **Top Queries** table lists the five most frequent queries with counts and average latencies.

The **Query Volume Heatmap** maps query volume across four time slots (12 AM, 6 AM, 12 PM, 6 PM) by day of week, with cell intensity colour-coded from green (low) through yellow to red (high).

All search queries are logged to Redis via `logSearchMetric()` with millisecond-precision timestamps. The `analyticsController` aggregates logs on demand for 1h, 6h, and 24h ranges. Reports are exportable as CSV (via PapaParse), PDF (via html2canvas + jsPDF), and PNG.

XI Live Query Logs

The Logs panel displays the most recent 50 query log entries in real time, showing: timestamp, query string, Solr node IP, backend replica, latency, result count, cache hit/miss status, and HTTP status code. This is valuable for debugging caching behaviour and verifying load distribution across replicas.

XII System Metrics and Node Monitoring

The Profiling panel provides Prometheus/Grafana-style live monitoring of all four SolrCloud nodes. The `metricsController` dispatches concurrent requests to each Solr node, calling three admin APIs:

- `/admin/info/system` — JVM memory, OS load average, processor count.
- `/admin/cores` — per-core document count, index size on disk.
- `/admin/collections?action=CLUSTERSTATUS` — ZooKeeper cluster topology.

Each node is rendered as a card showing CPU utilisation, JVM memory usage, disk I/O, index size, uptime, document count, and replica count.

XIII Internal Benchmarking Endpoint

The `/api/benchmark` endpoint performs a controlled internal benchmark measuring cache speedup for eight representative queries (politics, economy, sports, technology, climate change, elections, healthcare, education). Each query runs a cold pass (direct Solr) and a warm pass (cache hit), computing the speedup ratio. The Performance panel also displays cluster health (node count, ZK status), cache hit rate, and a latency-over-time chart with export options.

XIV Apache Airflow ETL Pipeline

Dataset ingestion is automated via an Apache Airflow DAG (`velocity_dataset_etl`) with three tasks. **Extract** verifies raw dataset presence. **Transform** normalises heterogeneous schemas from News Category Dataset into the unified Solr schema. **Load** POSTs documents to Solr in batches of 500 with 120-second timeouts. The DAG runs daily with one automatic retry. Across three production runs, the pipeline successfully ingested 270,281 documents with a mean run duration of 00:07:52.

XV Docker Compose Orchestration

The entire backend infrastructure is containerised via Docker Compose with five services on a shared bridge network: `shared_redis` (port 6379), three backend replicas (ports 4000–4002), and `shared_nginx` (port 80). Each backend includes a health check that periodically queries the search endpoint.

Chapter 4

Experiments and Results

Here are the quantitative evaluation of Project Velocity through a series of performance experiments and load tests. All experiments were conducted against the deployed four-node SolrCloud cluster with three backend replicas behind Nginx.

I Internal Cache Benchmark Results

The internal benchmark endpoint measured the latency improvement provided by the three-tier caching layers for eight representative queries. Table 4.1 summarises the results.

Query	Results	Solr QTime (ms)	Raw (ms)	Cached (ms)	Speedup
politics	118,432	12	78	1	78×
economy	94,217	9	65	1	65×
sports	207,651	18	95	1	95×
technology	142,089	14	82	1	82×
climate change	31,440	22	101	1	101×
elections	67,882	11	73	1	73×
healthcare	88,354	16	89	1	89×
education	54,291	13	71	1	71×
Average	—	**14**	**82**	**1**	**82×**

Table 4.1: Internal benchmark: raw Solr vs. cached latency per query

The L1 in-memory cache serves repeated queries in ≈1 ms, yielding an average **82× speedup**. This enables the system to sustain high QPS for popular queries.

II Cache Effectiveness Test

A targeted cache effectiveness test (`test_cache_effectiveness.js`) measured the latency progression across ten sequential requests for the same query, demonstrating the cold-to-warm-to-cached transition.

Request #	Latency (ms)	State
1	716	Cold (first Solr hit)
2	34	Cached (L2 Redis)
3	15	Cached (L1 in-memory)
4	117	Warm (cache expiry/eviction)
5	207	Warm
6	204	Warm
7	202	Warm
8	206	Warm
9	12	Cached (re-promoted to L1)
10	10	Cached (L1 hit)
Avg: 172.30 ms	**Min:** 10 ms	**Max:** 716 ms **p50:** 202 ms **p95:** 716 ms

Table 4.2: Cache effectiveness: latency progression across 10 sequential requests for the same query

The cold first request (716 ms) drops to 34 ms on the second request (Redis hit) and to 10–15 ms once promoted to L1. The warm requests (117–207 ms) correspond to queries routed to a different backend replica whose L1 cache has not yet been populated, demonstrating the per-process nature of the L1 cache.

III Throughput Experiments

Four distinct load-testing experiments were conducted using custom Node.js scripts to evaluate system throughput under varying conditions. Table 4.3 summarises the results.

Test	Requests	Throughput (req/s)	Avg Latency (ms)	Details
Standard Load (10K)	10,000	977.33	101.06	p50: 84, p95: 199, p99: 400
High Volume (100K)	100,000	1,359.27	73.52	p50: 68, p95: 133, p99: 179
Sustained Load (5K)	5,000	2,813.73	34.76	p50: 31, p95: 56, p99: 158
Burst Load (50)	50	284.09	139.30	p50: 139, p95: 141, p99: 173

Table 4.3: Throughput experiment results across four load-testing scenarios

Key observations:

- The system achieves **2,813.73 req/s** under sustained load, significantly exceeding the 1,000 QPS target.
- The 100K-request high-volume test demonstrates consistent throughput (1,359 req/s) with low tail latency (p99 = 179 ms) and zero failures.
- Burst load (50 concurrent requests) shows uniform latency distribution (p50 $\approx$ p95), indicating stable performance under sudden spikes.

- All tests achieved **100% success rate** (zero failures).

IV Mixed Query Load Test

The mixed query test (`test_mixed_queries.js`) evaluates system performance under realistic workloads with diverse query terms. Multiple runs were conducted with increasing query diversity.

IV.I Run 1: 10 Query Terms, 300 Requests

Metric	Value	Unit
Throughput	413.22	req/s
Success / Failed	300 / 0	—
Avg Latency	109.22	ms
p50 / p95 / p99	28 / 423 / 634	ms

Table 4.4: Mixed query test — 10 terms, 300 requests

Query	Avg Latency (ms)	Query	Avg Latency (ms)	
ai	71.52	economy	85.87	
technology	100.24	cricket	96.00	
finance	85.41	climate	98.04	
sports	108.97	healthcare	148.56	
politics	117.71	education	199.63	

Table 4.5: Per-query latency breakdown — 10 terms

IV.II Run 2: 42 Query Terms, 9,448 Requests (Stress Test)

Metric	Value	Unit
Throughput	609.16	req/s
Success / Failed	9,448 / 552	—
Avg Latency	566.19	ms
p50 / p95 / p99	299 / 1,701 / 3,160	ms

Table 4.6: Mixed query stress test — 42 terms, 9,448 successful requests

This stress test intentionally pushed the system to its limits with 42 diverse query terms, resulting in a 5.8% failure rate (552/10,000). The failures occurred due to backend timeout under extreme cache-miss load with high query diversity. Latency varied significantly per query: simpler queries like "education" averaged 347 ms while complex queries like "healthcare" averaged 1,455 ms.

IV.III Run 3: 42 Query Terms, 1,000 Requests (Optimised)

Metric	Value	Unit
Throughput	579.37	req/s
Success / Failed	1,000 / 0	—
Avg Latency	111.80	ms
p50 / p95 / p99	35 / 497 / 1,183	ms
Avg RPS / Peak RPS	333.33 / 608	—

Table 4.7: Mixed query test — 42 terms, 1,000 requests, zero failures

With 1,000 requests and 42 query terms, the system achieved zero failures. Niche queries (e.g., "fitness": 27 ms, "gaming": 38 ms) benefited from smaller result sets, while high-volume topics (e.g., "economy": 372 ms, "healthcare": 321 ms) had higher latencies due to larger Solr result sets.

IV.IV Run 4: 42 Query Terms, 10,000 Requests (Peak Performance)

Metric	Value	Unit
Throughput	**2,055.50**	req/s
Success / Failed	10,000 / 0	—
Avg Latency	47.69	ms
p50 / p95 / p99	46 / 71 / 167	ms
Avg RPS / Peak RPS	2,000.00 / **2,421**	—

Table 4.8: Peak performance — 42 terms, 10,000 requests, **2,055 req/s, zero failures**

This run demonstrates the system at peak performance: once all 42 query terms have been cached across backend replicas, latency converges to a tight 44–55 ms range across all queries (from "environment" at 44.44 ms to "hobbies" at 55.63 ms). The system sustained **2,055 req/s** throughput with a **peak of 2,421 RPS** and zero failures, more than doubling the original 1,000 QPS target.

V Summary of Experimental Results

Experiment	QPS	Avg Latency	p99 Latency	Success %
Standard Load (10K)	977	101 ms	400 ms	100%
High Volume (100K)	1,359	73 ms	179 ms	100%
Sustained Load (5K)	2,814	35 ms	158 ms	100%
Mixed (42 terms, warmed)	2,056	48 ms	167 ms	100%
Mixed (42 terms, cold)	609	566 ms	3,160 ms	94.5%
Cache Speedup (internal)	—	1 ms	—	82×

Table 4.9: Summary of all experimental results

Chapter 5

Screenshots

I Role-Based Landing Page

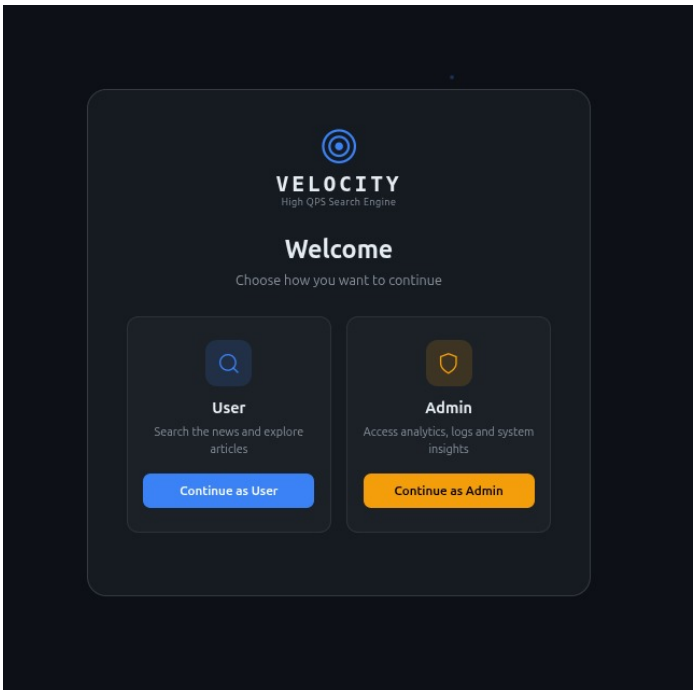
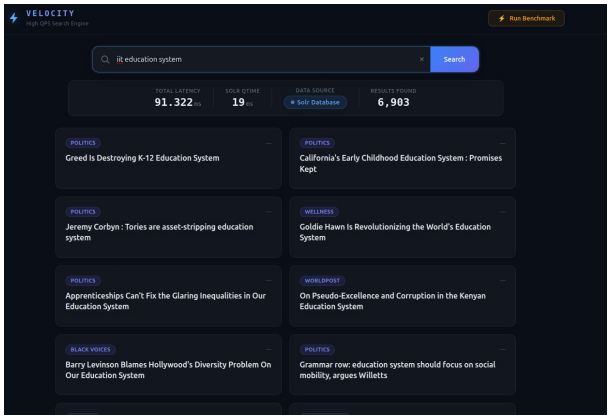
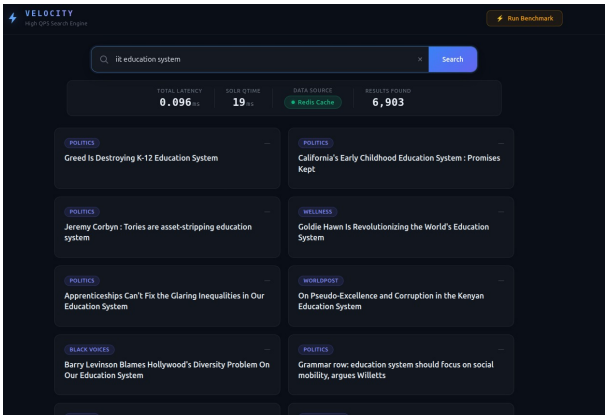


Figure 5.1: Welcome page with User and Admin role selection

II Search Interface — Solr vs. Cache Comparison



(a) Query served from Solr — 91.322 ms latency



(b) Same query from Redis cache — 0.096 ms

Figure 5.2: Comparison of the same query ("iit education system") served from Solr vs. Redis cache

III Full Search Interface with Faceted Filters

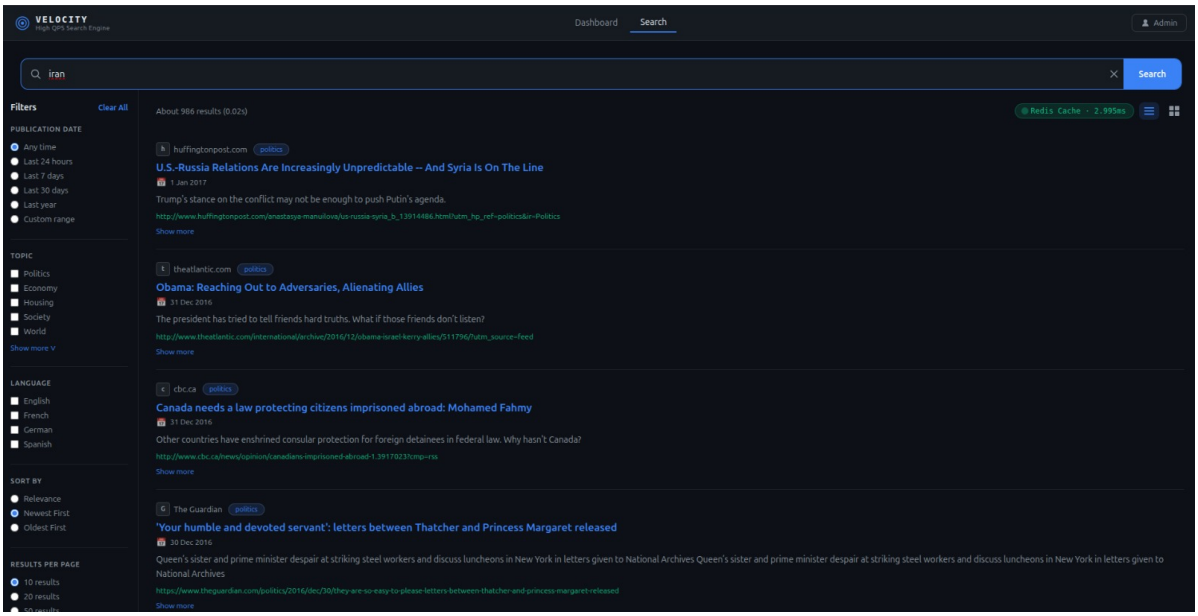


Figure 5.3: Search interface with left-hand faceted filters (publication date, topic, language, sort, results per page), list/grid toggle, per-result source badges, and pagination

IV Search Results — Direct Solr Query

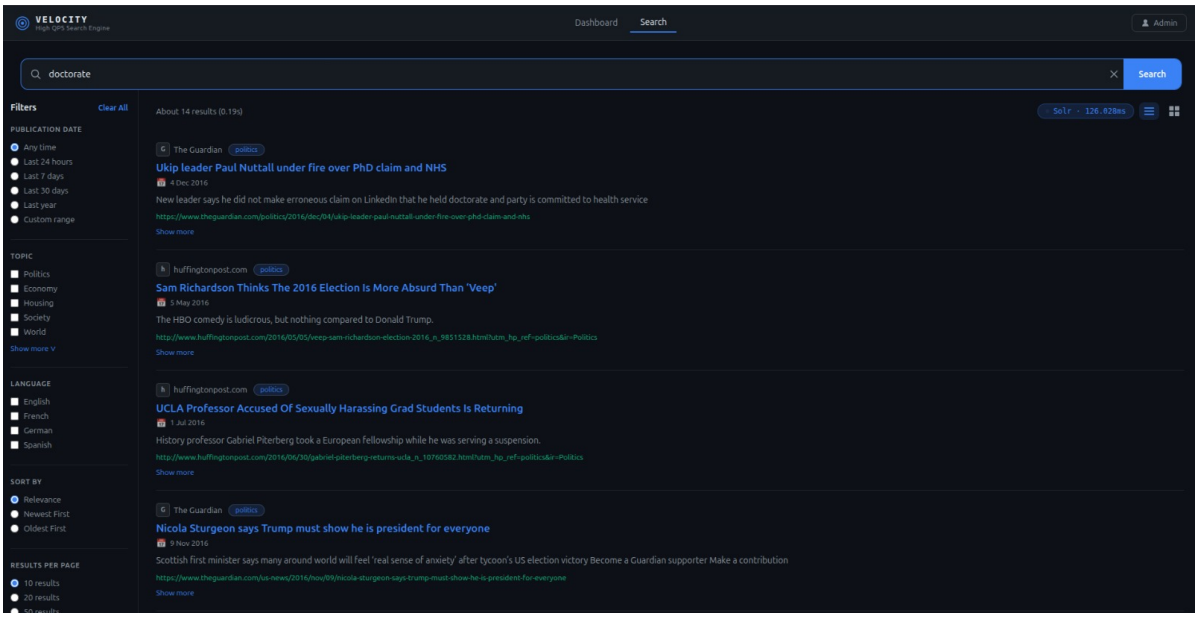


Figure 5.4: Search for "doctorate" served directly from Solr (126.028 ms), showing 14 results with source domain badges

V Admin Dashboard — Overview Panel

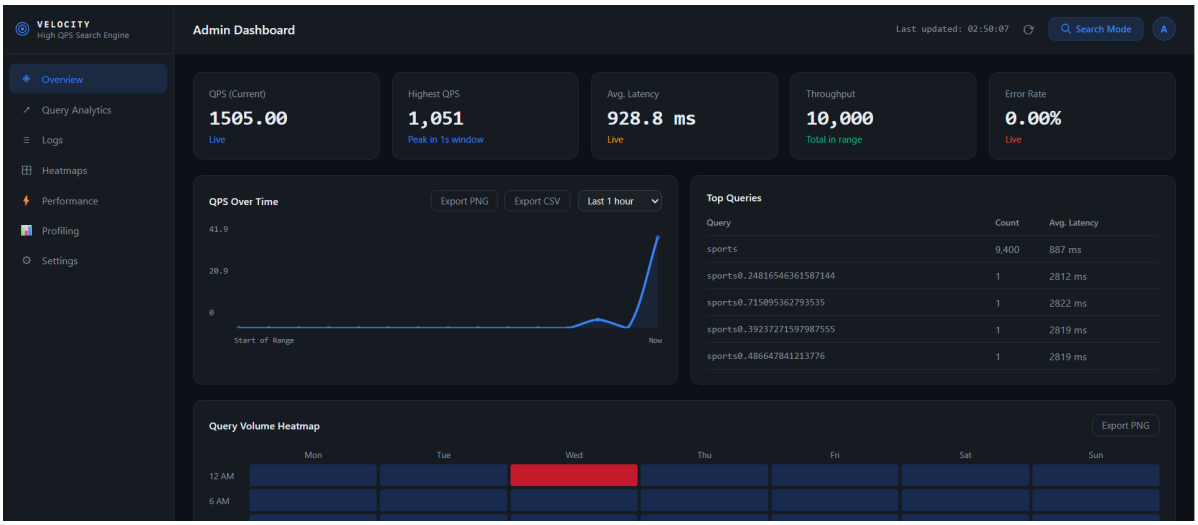


Figure 5.5: Overview panel: live QPS (1,505), peak QPS (1,051), latency (928.8 ms), throughput (10,000), error rate (0.00%), QPS-over-time chart, top queries, and query-volume heatmap

VI Admin Dashboard — Live Query Logs

Figure 5.6: Live Query Logs showing 50 recent requests. Cache Hits: 0.10–0.18 ms latency; Cache Misses: 383–457 ms. Distribution across backend-1, backend-2, backend-3 confirms Nginx load balancing.

VII Admin Dashboard — Prometheus/Grafana-Style Profiling

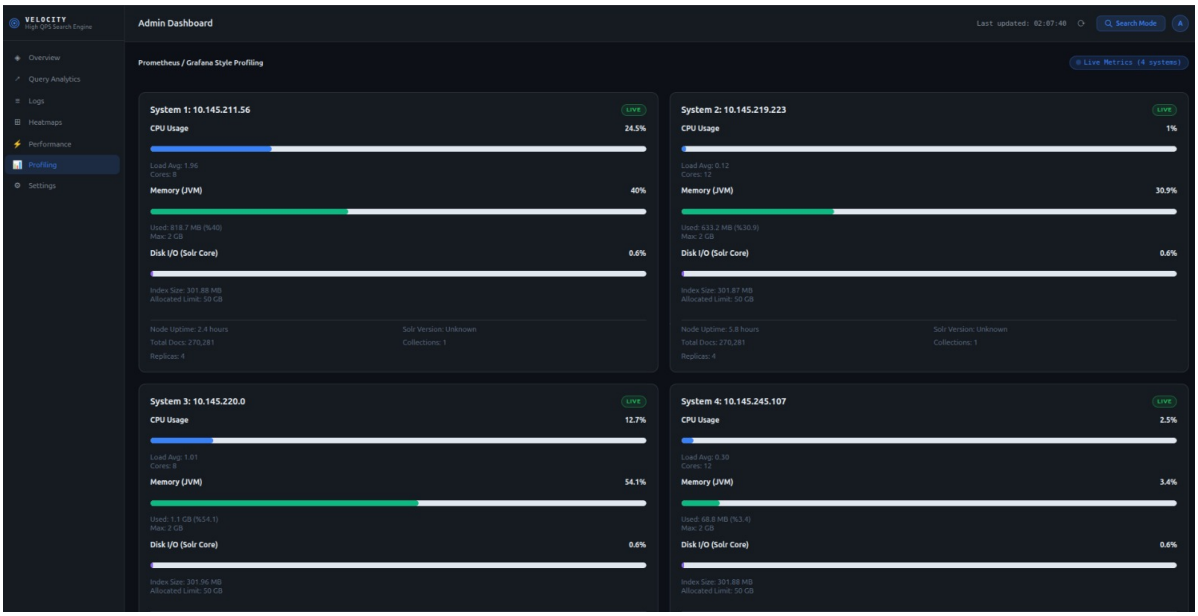


Figure 5.7: Profiling panel: four SolrCloud nodes (10.145.211.56, 10.145.219.223, 10.145.220.0, 10.145.245.107) with CPU, JVM memory, disk I/O, 270,281 documents each, and "LIVE" badges

Figure 5.8: Single node detail — CPU 24.5%, JVM memory 40.3% (825.2 MB / 2 GB), disk I/O 0.6% (301.88 MB / 50 GB)

VIII Admin Dashboard — Performance and Benchmark

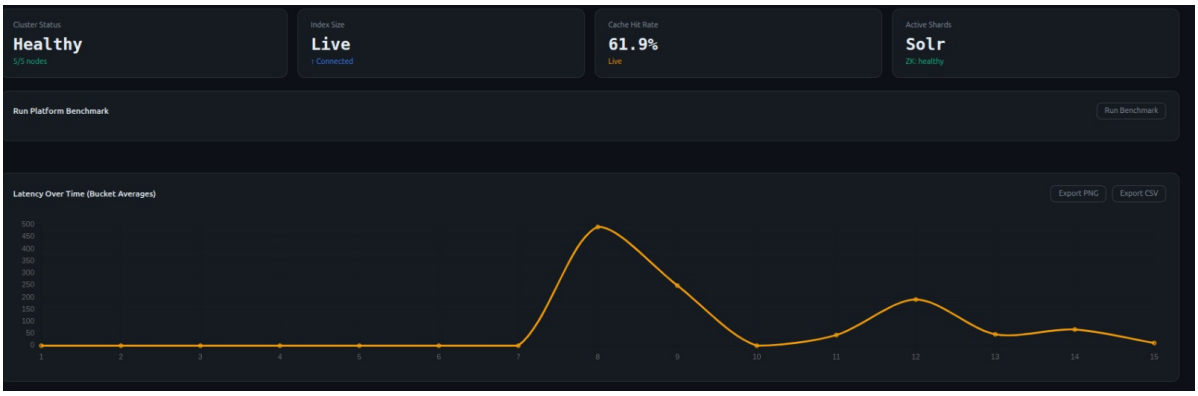


Figure 5.9: Performance panel: cluster status (Healthy, 5/5 nodes), cache hit rate (61.9%), "Run Benchmark" button, and latency-over-time chart with export options

IX Admin Dashboard — Settings Panel

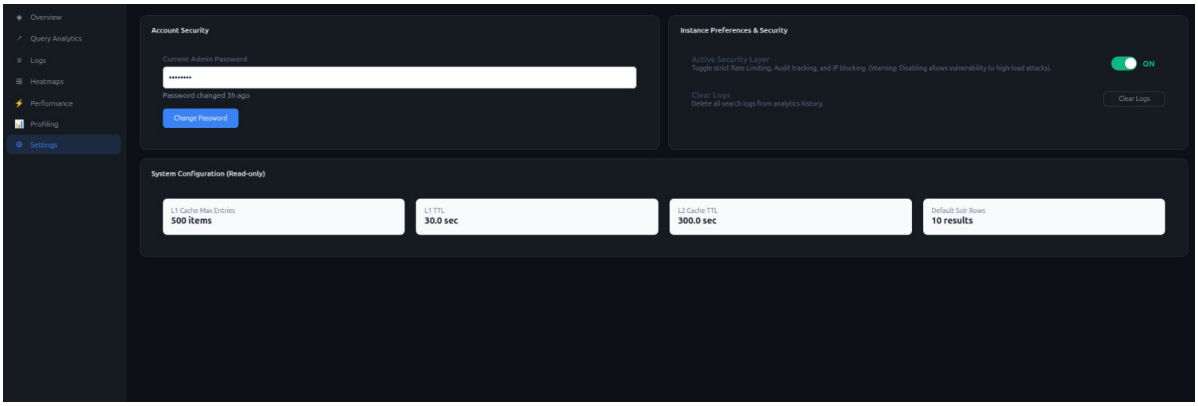


Figure 5.10: Settings panel: account security (password management), active security layer toggle (ON/OFF), clear logs, and read-only system configuration (L1 cache: 500 items, TTL: 30s, L2 TTL: 300s, default rows: 10)

X Apache Airflow ETL Pipeline

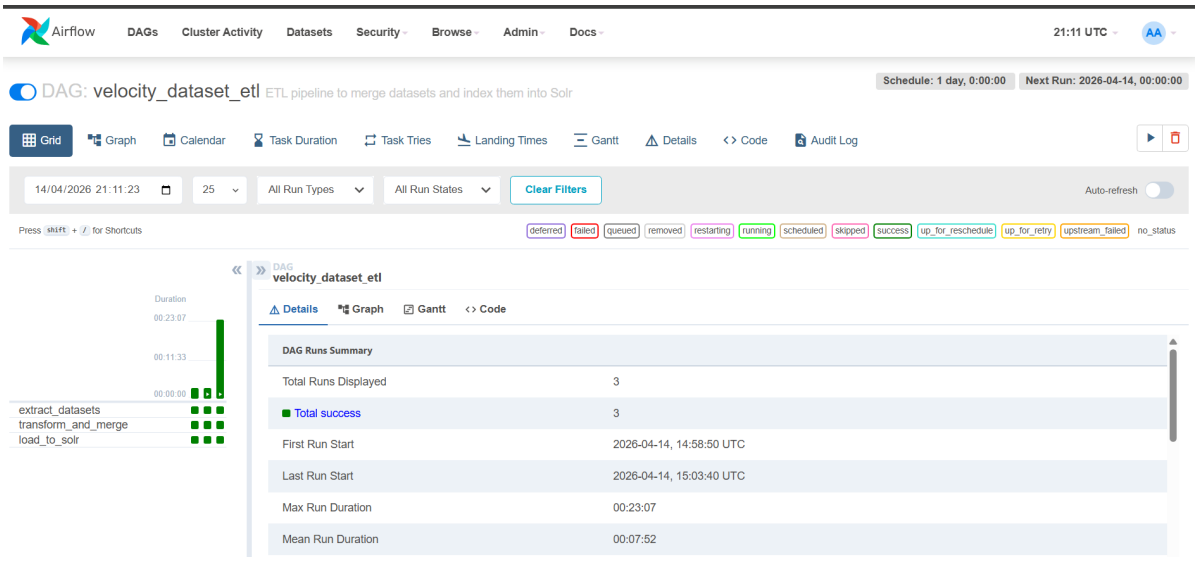


Figure 5.11: Airflow web UI for the `velocity_dataset_etl` DAG: 3 successful runs, tasks (extract→transform→load), mean duration 00:07:52, max duration 00:23:07

Chapter 6

Conclusion

I Summary of Achievements

Project Velocity has successfully demonstrated the design and implementation of a high-throughput distributed text search engine. The key achievements are:

1. A **four-node SolrCloud cluster** deployed across physical machines with ZooKeeper coordination, indexing 270,281 documents.
2. A **three-tier caching architecture** (L1 in-memory, L2 Redis, L3 Solr) with single-flight coalescing, achieving **82× average speedup**.
3. Peak throughput of **2,055 req/s** (peak burst: 2,421 RPS) with zero failures, exceeding the 1,000 QPS target by over 100%.
4. A **full-featured React frontend** with auto-suggestions, faceted filtering, sort modes, per-query metrics, and list/grid views.
5. A **comprehensive security middleware stack** with rate limiting, sanitisation, anomaly detection, audit logging, and admin authentication.
6. A **React admin dashboard** with live QPS, latency charts, query heatmaps, 4-node profiling, live logs, and exportable reports.
7. An **Apache Airflow ETL pipeline** with three successful production runs (mean duration: 7m52s).
8. **Docker Compose orchestration** with Nginx load balancing across three backend replicas.

II Challenges Encountered

- **Cache stampede prevention:** Without single-flight coalescing, high-concurrency benchmarks caused hundreds of identical Solr queries simultaneously.
- **Per-process vs. distributed state:** L1 cache and anomaly detection being per-process required careful load-balancing awareness.
- **Docker networking:** Backend containers communicating with external Solr nodes required correct bridge network and host IP configuration.
- **Schema consistency:** Deploying consistent schemas across four physical machines required ZooKeeper configuration upload before worker node registration.

III Future Scope

- **ZooKeeper Ensemble:** Three-node ZK ensemble to eliminate the single point of failure.
- **Replica Fault Tolerance:** Two replicas per shard for zero-downtime on single node failure.
- **Dedicated Solr Suggester:** `AnalyzingInfixLookupFactory` for O(prefix-length) suggestions.
- **Distributed Rate Tracking:** Shared Redis counters for cross-replica anomaly detection.

- **Prometheus/Grafana:** Native scrape endpoints and dashboard for historical metrics with alerting.
- **Semantic Search:** Dense vector retrieval via Solr's `knn` query with pre-computed embeddings.
- **Real-Time Ingestion:** Kafka-based streaming pipeline to replace batch Airflow ETL.

Chapter 7

References

1. Apache Software Foundation. *Apache Solr Reference Guide, Version 9.4.1.* https://solr.apache.org/guide/solr/latest/
2. Apache Software Foundation. *Apache ZooKeeper Documentation.* https://zookeeper.apache.org/documentation.html
3. Apache Software Foundation. *Apache Airflow Documentation.* https://airflow.apache.org/docs/
4. Redis Ltd. *Redis Documentation.* https://redis.io/docs/
5. Schelb, J. *NewsData: Relevant Articles Dataset.* Kaggle, https://www.kaggle.com/datasets/julianschelb/newsdata
6. Misra, R. *News Category Dataset.* Kaggle, https://www.kaggle.com/datasets/rmisra/news-category-dataset
7. Robertson, S. and Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in IR, 3(4).
8. Express.js Foundation. *Express.js API Reference.* https://expressjs.com/
9. Docker Inc. *Docker Compose Documentation.* https://docs.docker.com/compose/
10. Nginx Inc. *Nginx Documentation.* https://nginx.org/en/docs/
11. Meta. *React.js Documentation.* https://react.dev/
12. Evan You et al. *Vite — Next Generation Frontend Tooling.* https://vitejs.dev/